Grupa: _____ Numele și prenumele (cu MAJUSCULE): _____

Arhitectura Sistemelor de Calcul: Parțial – 10% din nota finală

Toate foile suplimentare trebuie semnate!

Exercițiul 1 **1.5p** Scrieți dimensiunea alocată în RAM, atât în biți, cât și în Bytes, pentru următoarele variabile declarate în secțiunea `.data`:

Declarare în .data	**dimensiune în biți**	**dimensiune în Bytes**
`str1: .ascii "Sir de\n"`		
`str2: .asciz "caractere\n"`		
`myWord: .word 13, 25, 260`		
`myLong: .long 0x8fde2025`		
`myBytes: .byte 't', 'e', 's', 't'`		

Scrieți un apel de sistem `WRITE` (codul apelului este 4) pentru șirul de caractere `str1`, astfel încât să afișați, la standard output, întreg conținutul șirului de caractere și nimic altceva în plus.

Exercițiul 2 **1p** Completați următorul tabel, cu valorile corecte în bazele 4 și 16.

Baza 4	**Baza 16**
	`04 00 84 02`
23 10 12 20 00 00 33 30 00 10	

Exercițiul 3 **2p** Fie următorul program, scris în assembly (procesor Intel x86, sintaxa AT&T):

```
.data
    x: .long 0x04008402
    z: .long 0x20000
    t: .long 0b1 0000 0000 0000
.text
.global main
main:
    movl x, %eax
    mull z
et1:
    divl t
et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Executăm, în **debugger**, următoarele comenzi:

`b main; b et_exit; run; stepi; stepi; i r eax; i r edx; c; i r eax; i r edx;`

Valorile pe care le vom obține sunt, în ordine: **Important!** Completați valorile exclusiv în baza 16.

EAX (1)	EDX (1)	EAX (2)	EDX (2)

Exercițiul 4 **1.5p** Fie următoarele instrucțiuni în `main`:

```
movl $0x2000 0000, %eax
movl $0x0040 0000, %ebx
mul %ebx
divl p
```

Descrieți ce obținem dacă $p = 2^9$. Dar dacă $p = 2^{10}$?

Exercițiul 5 **1p** Fie următorul program, scris în assembly (procesor Intel x86, sintaxa AT&T):

```
.data
 x: .long 10
 y: .long 50
.text
.global main
main:
    movl x, %ebx
    movl y, %eax
    movl $0x10, %edx
    mul %ebx
    add %edx, %eax
    movl $4, %esi
    divl %esi
    movl %edx, %ecx
    addl $35, %ecx
et_loop:
    inc %ebx
    inc %edx
    dec %eax
    loop et_loop
    add %edx, %eax
et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
```

Completați valorile regiștrilor (**în baza 10**) în momentul în care execuția programului ajunge în dreptul etichetei `et_exit` (după ce am rulat în `debugger` comenzile `b et_exit; run; i r`):

EAX	EBX	EDX

Exercițiul 6 **1p** Fie următorul program, dezvoltat în limbajul de asamblare al procesorului Intel x86, sintaxa AT&T:

```
.data
.text
.global main
main:
    movl $0x8788df0b, %ecx
    sub %eax, %eax
    sub %ebx, %ebx
    movl $0, %edx
    movw %cx, %dx
    movb %dh, %al
    movb %dl, %bh
    addw %ax, %bx
    cmp %ebx, %edx
    jg et
et_loop:
    cmp $0, %eax
    je et_exit
    dec %ebx
    dec %eax
    jmp et_loop
et_exit:
    movl $1, %eax
    xorl %ebx, %ebx
    int $0x80
et:
    cmp $0, %eax
    je et_exit
    inc %ebx
    dec %eax
    jmp et_loop
```

Valorile stocate în dreptul etichetei `et_exit` (în baza 16) sunt:

EAX	EBX	ECX	EDX

Exercițiul 7 **1p** Putem determina eticheta de memorie a etichetelor dintr-un program? Dacă da, cum, iar dacă nu, de ce? Dar putem modifica adresele etichetelor? Dacă da, cum, iar dacă nu, de ce?

Exercițiul 8 **1p** Scrieți o secvență de cod în `main` care să calculeze maximul elementelor impare dintr-un `array` de elemente de tip `.long`. Considerați că lungimea array-ului este `n`, numele lui este `v` și toate elementele sunt pozitive. Se garantează că cel puțin un element este impar. Aceste informații sunt deja în secțiunea `.data`. Soluția voastră trebuie să înceapă de la eticheta `main`.